

Code Review — victor-vectori

Toate headerele din `vectori/` + `tema4.h` · punct de intrare: `vectori.cpp` → `tema4.h` → `solutie2g()` · 21.07.2026

Codul NU a fost modificat — toate observațiile sunt doar aici. Priorități: ● bug real · ● design/pattern · ● stil. Soluțiile complete (problema / mecanism / cod corect) sunt în documentul companion **SOLUTII_REVIEW**, cu aceeași numerotare.

● Critice — bug-uri reale

Firul comun (B1–B7): `d` nu corespunde datelor reale, sau bucla atinge un vecin inexistent. Regula: **existența unui vecin o decide indexul, nu valoarea lui.**

B1 — `d = 5` pe un vector cu un singur element

`dublete.h:29–34` (`solutie1a`)

`int v[100] = { 5 }; int d = 5;` — pozițiile 1–4 sunt 0 dar declarate ca date; în plus `i < d` cu `v[i+1]` citește `v[5]`. Pentru dublete: `i < d - 1`.

B2 — `d = 8` pe un vector cu 6 elemente

`tema3.h:635–636`

`{ 8, 9, 9, 4, 5, 7 }` cu `d = 8` → pozițiile 6,7 (=0) intră în sortare ca elemente reale.

B3 — `d = 6` pe un vector cu 4 elemente

`tema4.h:235–236` (`solutie1g`)

`{ 2, 3, 5, 7 }` cu `d = 6` → pozițiile 4,5 (=0) intră în `cmmdc`.

B4 — off-by-one la dublete

`realizareaUnuiNouVector.h:364–365` (`solutie2b`)

`i < d` cu `v[i+1]` → la `i = d-1` citește `v[d]` (=0). Rezultat fals *silentios*, mai periculos decât un crash. Corect: `i < d - 1`.

B5 — off-by-one la `solutie2c`

`tema4.h:354–356`

`for (i = 0; i < d; i++)` cu `v[i+1]` → la `i = d-1` citește `v[d]`. Corect: `i < d - 1`.

B6 — $v[i-1]$ la $i == 0$ — graniță pusă pe valoare

corectareTema2.h:109

`if (v[i + 1] != 0 && v[i - 1] >= 1) : la  $i = 0$ ,  $v[i-1]$  este  $v[-1]$  (UB). Verifică pe index: if (i > 0 && i < d - 1).`

B7 — ștergere cu index -1 în bucla de căutare

tema4.h:60-70 (solutie1a)

`stergereElement(v, d, poz)` și `afisare` sunt în buclă; cât timp `poz == -1`, `stergereElement(v, d, -1)` scrie $v[-1]$ (UB) și distruge datele. Șterge o singură dată, **după** buclă, doar dacă `poz != -1`. Idem `stergere_inserare.h:75-84`.

B8 — operatorul virgulă în loc de cmmdc

corectareTema2.h:249 · tema2.h:655

`return a * b / (a, b);` — `(a, b)` e operatorul virgulă ($\rightarrow b$), deci expresia devine `a`. Corect: apel real `cmmdc(a, b)`.

B9 — comparație de parități înlănțuită

corectareTema2.h:110

`v[i]%2 == v[i-1]%2 == v[i+1]%2` se evaluează ca `(0/1) == v[i+1]%2`. Pentru trei pare dă `false`. Corect: legat cu `&&`.

B10 — cmmdc prin scăderi: buclă infinită pentru 0

realizareaUnuiNouVector.h:59-70

`while (a != b)` cu scăderi nu se termină la `cmmdc(0, 45)`. Declanșat real în `solutie3a`. Folosește Euclid cu rest (ai deja în `dublete.h:18-26`).

B11 — sortezi aux, afișezi v

sortareaVectorilor.h:108-109

`sortatPrinBubbleSort(aux, ct); afisare(v, d);` — afișezi originalul neatins. Corect: `afisare(aux, ct)`.

🟡 Importante — design / pattern

M1 — funcții definite duplicat în headere

prim × 7 fișiere, + afisare, cmmdc, stergereElement...

La primul include dublu → „function redefinition”. `#pragma once` nu apără de ciocnirea între fișiere diferite.

Header = declarații; definițiile în `.cpp`. Pragmatic: un singur `utile.h`.

M2 — poziție -1 folosită ca index de start

MunteVale.h:119–126

`deterPozitie1Ex1` poate întoarce -1, apoi `sortatDescrescatorInterval(v, -1, d-1) → v[-1]`. Verifică `if (poz == -1) return;`.

M3 — 6 valori în inițializare, d = 5

realizareaUnuiNouVector.h:418–419 (solutie2d)

„Merge” din motivul greșit (`i < d + v[i+1]` prinde al 6-lea element). `d = 6 + i < d - 1`.

M4 — baza2 acumulează în int deși promite long long

tema2.h:230–241

Pentru `n ≥ 1024` depășirea se produce în `int b2 / int p` înainte de return. Variabilele de lucru trebuie `long long`.

M5 — % 2 == 1 pică pe numere negative

exercitiiVectori.h:36

`-7 % 2` este `-1`, nu `1`. Idiom robust: `v[i] % 2 != 0`.

M6 — santinele magice în loc de primul element

realizareaUnuiNouVector.h:505,520,763,776

`min = 9999` eșuează dacă toate elementele sunt peste 9999. Pornește cu `v[0]`, compară de la `i = 1`.

M7 — solutie3b neterminată

tema2.h:529

Corp gol (`int desc = 0;`). Descompunerea există în `corectareTema2.h:79` — de refolosit.

M8 — tema4.h reintroduce funcțiile deja definite

tema4.h · `stergere_inserare.h`

`prim`, `afisare`, `stergereElement`, `inserareElement`, `oglindit` ... redefinite. Extinde M1.
`stergere_inserare.h` e duplicat parțial.

Cleanups — stil

C1 — bubble sort rescris de ~8 ori

tema3.h:8,182,356,460,494,550...

Schelet identic, diferă doar criteriul. O pereche crescător/descrescător refolosită + criteriu calculat separat.

C2 — if (cond) return true; return false; → return cond;

tema2.h · `realizareaUnuiNouVector.h` · `corectareTema2.h` · `tema4.h` (`nrNegativ`, `nul`, `patratPerfect`, `trei_Div...`)

Condiția e deja bool-ul dorit.

C3 — bool comparat cu numere / bool ca 0/1

tema2.h:176 · `corectareTema2.h:51` · `tema4.h:190,212`

`bool vf = 1 → true; prim(n) == 0 → !prim(n); while (sortat == false) → while (!sortat).`

C4 — nume înșelătoare

tema3.h:282 · `sortareaVectorilor.h:19` · `tema2.h:108` · `tema4.h:106`

`bubbleSort...` / `sortareaPrinSelectie` sunt de fapt exchange sort; `primCifEgalultimCif` capitalizare inconsecventă; `nul(n)` wrapper trivial.

C5 — cod mort

tema2.h:651,641 · MunteVale.h:132-155 · corectareTema2.h:234 · realizareaUnuiNouVector.h:375 · exercitiiVectori.h:49

```
cmmmc neapelat, baza2 / ctCifre1 nefolosite, int poz nefolosit, int aux nefolosit, ;; dublu.
```

C6 — patratPerfect cu comparație float

tema3.h:278 · corectareTema2.h:66

```
sqrt(n) == (int)sqrt(n) fragil pe valori mari. Folosește int r = (int)round(sqrt(n)); return r*r == n; (ai deja în tema2.h:514).
```

C7 — min/max ca nume proprii sub using namespace std

realizareaUnuiNouVector.h:762,775

Coliziune cu std::min / std::max; minim / maxim erau mai sigure.

Before / After — criticele

#	Before (actual)	After (corect)
B1	<pre>int v[100] = { 5 }; int d = 5; ... v[i+1]</pre>	<pre>v = { 5, 8, 9, 14, 25 }; for (i=0; i<d-1; i++)</pre>
B2	<pre>int d = 8; // 6 valori</pre>	<pre>int d = 6;</pre>
B3	<pre>int d = 6; // 4 valori</pre>	<pre>int d = 4;</pre>
B4	<pre>for (i=0; i<d; i++) cu v[i+1]</pre>	<pre>for (i=0; i<d-1; i++)</pre>
B5	<pre>for (i=0; i<d; i++) cu v[i+1]</pre>	<pre>for (i=0; i<d-1; i++)</pre>
B6	<pre>if (v[i+1] != 0 && v[i-1] >= 1)</pre>	<pre>if (i > 0 && i < d - 1)</pre>
B7	<pre>în buclă: stergereElement(v,d,poz); afisare(v,d);</pre>	<pre>după buclă: if (poz != -1) stergereElement(v,d,poz); afisare(v,d);</pre>
B8	<pre>return a * b / (a, b);</pre>	<pre>return a * b / cmmdc(a, b);</pre>
B9	<pre>v[i]%2 == v[i-1]%2 == v[i+1]%2</pre>	<pre>v[i]%2 == v[i-1]%2 && v[i-1]%2 == v[i+1]%2</pre>
B10	<pre>while (a != b) { if(a>b) a-=b; if(b>a) b-=a; }</pre>	<pre>while (b != 0) { int r=a%b; a=b; b=r; }</pre>
B11	<pre>sortatPrinBubbleSort(aux, ct); afisare(v, d);</pre>	<pre>sortatPrinBubbleSort(aux, ct); afisare(aux, ct);</pre>

Q&A — verifică-ți înțelegerea

1. Ce valoare are expresia `(a, b)` în C++ și de ce `a * b / (a, b)` compilează fără eroare, deși calculează greșit?
2. Pentru trei numere pare, ce valoare are `v[i]%2 == v[i-1]%2 == v[i+1]%2` pas cu pas? De ce rezultatul e `false` exact în cazul „toate au aceeași paritate”?
3. De ce cmmdc-ul prin scăderi repetate se blochează când unul dintre numere e 0, iar Euclid cu `%` nu? Urmărește ambele pe hârtie cu (45, 0).
4. Dacă în `vectori.cpp` incluzi și `dublete.h` și `tema2.h`, ce eroare dă compilatorul și de ce `#pragma once` nu te ajută aici?
5. În `solutie1a (tema4.h)`, de ce `stergereElement(v, d, -1)` scrie în afara vectorului? Urmărește prima iterație a buclei interne cu `poz = -1`.
6. La `solutie2c`, de ce bucla `i < d` cu `v[i+1]` „nu crapă” deși citește `v[d]`, dar tot dă rezultat greșit? Care e diferența între „nu crapă” și „e corect”?